

Міністерство освіти і науки України
Національний технічний університет України
"Київський політехнічний інститут імені Ігоря Сікорського"
Фізико-технічний інститут

Аналіз кіберінцидентів
Практична робота №2

Виконали:
студенти гр. ФБ-62мп
Маврикін Едуард
Смірнова Наталія

Київ–2026

Хід роботи:

Тема: Дерево рішень і ліс дерев рішень в системах машинного навчання

Мета: застосувати алгоритми дерева рішень та випадкового лісу до трьох датасетів з кібербезпеки, проаналізувати переваги та недоліки алгоритмів і залежність якості класифікації від особливостей даних.

1. Теоретичні відомості

Дерево рішень – модель навчання з учителем, у якій кожен внутрішній вузол є логічним тестом над ознакою, гілка – результатом тесту, а лист – класом. Дерево будується рекурсивним розбиттям навчальної вибірки; у кожному вузлі обирається тест, що мінімізує середньозважену невпорядкованість дочірніх вузлів. Як міру використано індекс Джині:
$Gini(E) = 1 - \Sigma p_i^2$
Основний недолік дерева – схильність до перенавчання та нестабільність, тому глибину дерева обмежують або обрізають.

Випадковий ліс (Random Forest) – ансамбль дерев, побудований методом беггінгу: кожне дерево навчається на бутстреп-вибірці, а в кожному вузлі розглядається лише випадкова підмножина ознак. Підсумковий клас визначається голосуванням дерев, що зменшує дисперсію та ризик перенавчання.

2. Вихідні дані

Обрано три датасети Kaggle з різними особливостями:

Датасет	Записів	Ознаки	Клас 1
Malware Executable Detection	373	531, бінарні (hex-код + виклики DLL)	81 % malware
Phishing Websites	11 054	30, категоріальні (−1/0/1) ознаки URL і сторінки	44 % phishing
NSL-KDD	125 973 + 22 544	41, числові та категоріальні ознаки з'єднань	47 % атак

Malware та Phishing поділено на навчальну і тестову частини 75/25 зі стратифікацією. Для NSL-KDD використано офіційне розбиття KDDTrain+/KDDTest+: тестова частина містить 3750 записів 17 типів атак, яких немає в навчальній. Категоріальні ознаки NSL-KDD (protocol_type, service, flag) закодовано one-hot, отримано 122 ознаки.

3. Побудова моделей

Для кожного датасету навчено DecisionTreeClassifier без обмеження глибини та RandomForestClassifier зі 200 деревами. Оцінено accuracy на навчальній і тестовій вибірках, F1, ROC AUC та матриці спряженості. Додатково побудовано залежність точності дерева від max_depth і візуалізацію дерева глибини 3.

Код програми:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score,
ConfusionMatrixDisplay

KDD_COLS = [
    "duration", "protocol_type", "service", "flag", "src_bytes",
"dst_bytes",
    "land", "wrong_fragment", "urgent", "hot", "num_failed_logins",
"logged_in",
    "num_compromised", "root_shell", "su_attempted", "num_root",
    "num_file_creations", "num_shells", "num_access_files",
"num_outbound_cmds",
    "is_host_login", "is_guest_login", "count", "srv_count", "serror_rate",
    "srv_serror_rate", "rerror_rate", "srv_rerror_rate", "same_srv_rate",
    "diff_srv_rate", "srv_diff_host_rate", "dst_host_count",
"dst_host_srv_count",
    "dst_host_same_srv_rate", "dst_host_diff_srv_rate",
    "dst_host_same_src_port_rate", "dst_host_srv_diff_host_rate",
    "dst_host_serror_rate", "dst_host_srv_serror_rate",
"dst_host_rerror_rate",
    "dst_host_srv_rerror_rate", "attack", "level",
]


def load_malware():
    df = pd.read_csv("uci_malware_detection.csv")
    X = df.drop(columns="Label")
    y = (df["Label"] == "malicious").astype(int)
    return train_test_split(X, y, test_size=0.25, stratify=y,
```

```
random_state=42)


def load_phishing():
    df = pd.read_csv("phishing.csv")
    X = df.drop(columns=["Index", "class"])
    y = (df["class"] == -1).astype(int)
    return train_test_split(X, y, test_size=0.25, stratify=y,
random_state=42)


def load_kdd():
    train = pd.read_csv("KDDTrain+.txt", names=KDD_COLS)
    test = pd.read_csv("KDDTest+.txt", names=KDD_COLS)
    both = pd.get_dummies(
        pd.concat([train, test]).drop(columns="level"),
        columns=["protocol_type", "service", "flag"],
    )
    X = both.drop(columns="attack")
    y = (both["attack"] != "normal").astype(int)
    n = len(train)
    return X[:n], X[n:], y[:n], y[n:]


datasets = {"Malware": load_malware(), "Phishing": load_phishing(), "NSL-
KDD": load_kdd()}
models = {
    "Decision Tree": lambda: DecisionTreeClassifier(random_state=42),
    "Random Forest": lambda: RandomForestClassifier(n_estimators=200,
n_jobs=-1, random_state=42),
}

rows, fitted = [], {}
fig, axes = plt.subplots(2, 3, figsize=(13, 8))
for j, (dname, (X_train, X_test, y_train, y_test)) in
enumerate(datasets.items()):
    print(f"{dname}: train {X_train.shape}, test {X_test.shape}, частка
класу 1 {y_train.mean():.2f}")
    for i, (mname, make) in enumerate(models.items()):
        m = make().fit(X_train, y_train)
        fitted[dname, mname] = m
        pred = m.predict(X_test)
        rows.append({
            "Датасет": dname,
            "Модель": mname,
            "Train acc": accuracy_score(y_train, m.predict(X_train)),
            "Test acc": accuracy_score(y_test, pred),
            "F1": f1_score(y_test, pred),
            "ROC AUC": roc_auc_score(y_test, m.predict_proba(X_test)[:,
1]),
            "Глибина": m.get_depth() if mname == "Decision Tree" else
max(t.get_depth() for t in m.estimators_),
        })
```

```
        ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=axes[i,
j], colorbar=False)
        axes[i, j].set_title(f"{dname} – {mname}")
plt.tight_layout()
plt.savefig("confusion.png", dpi=110)

res = pd.DataFrame(rows).round(4)
print(res.to_string(index=False))

train_attacks = set(pd.read_csv("KDDTrain+.txt", names=KDD_COLS)["attack"])
test_attacks = pd.read_csv("KDDTest+.txt", names=KDD_COLS)["attack"]
new = ~test_attacks.isin(train_attacks).values
old = (test_attacks != "normal").values & ~new
X_test = datasets["NSL-KDD"][1]
for mname in models:
    pred = fitted["NSL-KDD", mname].predict(X_test)
    print(f"NSL-KDD {mname}: виявлено відомих атак {pred[old].mean():.3f},
нових {pred[new].mean():.3f} ({new.sum()} записів)")

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for ax, dname in zip(axes, datasets):
    X_train = datasets[dname][0]
    imp = pd.Series(fitted[dname, "Random Forest"].feature_importances_,
index=X_train.columns)
    imp.nlargest(10)[::-1].plot.barh(ax=ax, title=dname)
    print(f"Топ-5 ознак RF {dname}:", imp.nlargest(5).round(3).to_dict())
plt.tight_layout()
plt.savefig("importance.png", dpi=110)

X_train, X_test, y_train, y_test = datasets["Phishing"]
depths = range(1, 26)
tr, te = [], []
for d in depths:
    m = DecisionTreeClassifier(max_depth=d, random_state=42).fit(X_train,
y_train)
    tr.append(m.score(X_train, y_train))
    te.append(m.score(X_test, y_test))
plt.figure(figsize=(7, 4))
plt.plot(depths, tr, label="train")
plt.plot(depths, te, label="test")
plt.xlabel("max_depth")
plt.ylabel("accuracy")
plt.legend()
plt.tight_layout()
plt.savefig("depth.png", dpi=110)

small = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_train,
y_train)
plt.figure(figsize=(16, 7))
plot_tree(small, feature_names=X_train.columns, class_names=["legit",
"phishing"], filled=True, fontsize=9)
plt.savefig("tree.png", dpi=110, bbox_inches="tight")
```

4. Результати

Результат виконання програми:

```
Malware: train (279, 531), test (94, 531), частка класу 1 0.81
Phishing: train (8290, 30), test (2764, 30), частка класу 1 0.44
NSL-KDD: train (125973, 122), test (22544, 122), частка класу 1 0.47
 Датасет        Модель  Train acc  Test acc     F1  ROC AUC  Глибина
 Malware Decision Tree     1.0000    0.9894 0.9935   0.9722        2
 Malware Random Forest     1.0000    1.0000 1.0000   1.0000        7
Phishing Decision Tree     0.9897    0.9635 0.9582   0.9740       23
Phishing Random Forest     0.9897    0.9743 0.9707   0.9963       28
 NSL-KDD Decision Tree     0.9999    0.7853 0.7742   0.8078       21
 NSL-KDD Random Forest     0.9999    0.7677 0.7500   0.9611       41
NSL-KDD Decision Tree: виявлено відомих атак 0.755, нових 0.383 (3750
записів)
NSL-KDD Random Forest: виявлено відомих атак 0.768, нових 0.234 (3750
записів)
Топ-5 ознак RF Malware: {'F_20': 0.056, 'F_19': 0.053, 'F_139': 0.046,
'F_179': 0.042, 'F_140': 0.04}
Топ-5 ознак RF Phishing: {'HTTPS': 0.337, 'AnchorURL': 0.23,
'WebsiteTraffic': 0.076, 'SubDomains': 0.065, 'LinksInScriptTags': 0.042}
Топ-5 ознак RF NSL-KDD: {'src_bytes': 0.145, 'dst_bytes': 0.087, 'flag_SF':
0.084, 'dst_host_srv_count': 0.053, 'same_srv_rate': 0.052}
```

Датасет	Модель	Train acc	Test acc	F1	ROC AUC
Malware	DT	1.000	0.989	0.994	0.972
Malware	RF	1.000	1.000	1.000	1.000
Phishing	DT	0.990	0.964	0.958	0.974
Phishing	RF	0.990	0.974	0.971	0.996
NSL-KDD	DT	1.000	0.785	0.774	0.808
NSL-KDD	RF	1.000	0.768	0.750	0.961

Матриці спряженості для всіх датасетів і моделей наведено на рис.1.

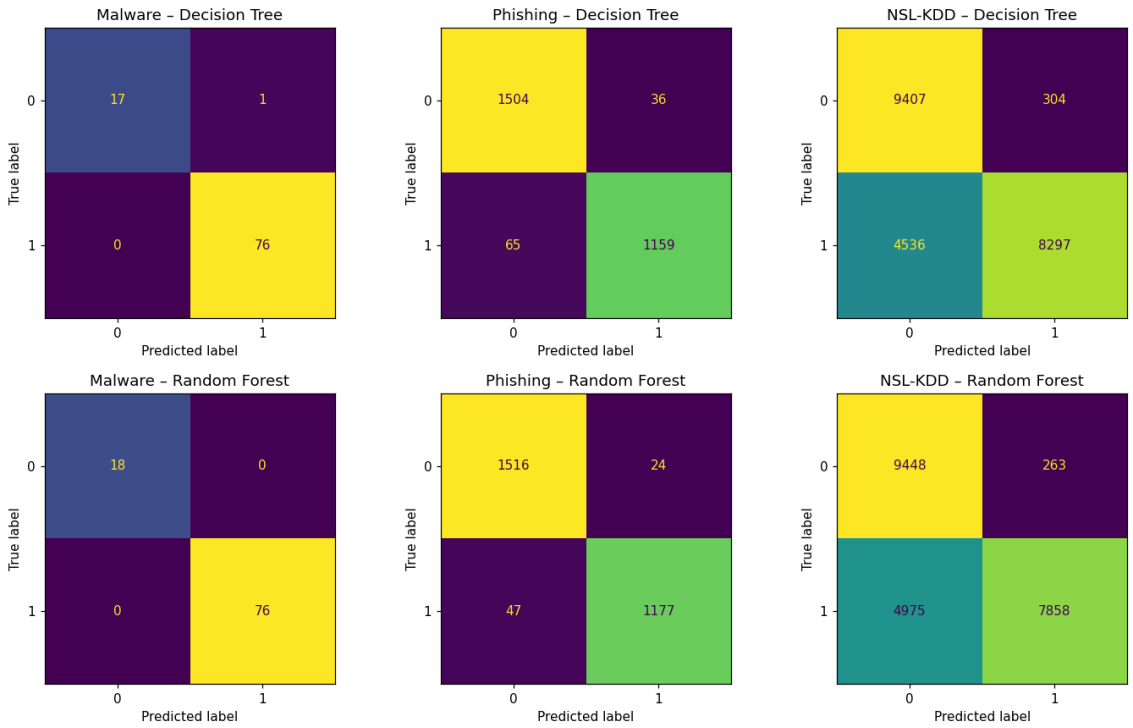


рис.1

Malware: дерево глибиною лише 2 майже ідеально розділяє класи (1 помилка), ліс класифікує всі 94 файли правильно. Дані малі, але добре роздільні.

Phishing: ліс перевершує дерево за всіма метриками (accuracy 0.974 проти 0.964, ROC AUC 0.996 проти 0.974). Різниця між train і test accuracy вказує на перенавчання необмеженого дерева (глибина 23). На рис.2 видно, що після глибини ≈ 15 точність на тесті перестає зростати, тоді як на навчальній вибірці росте.

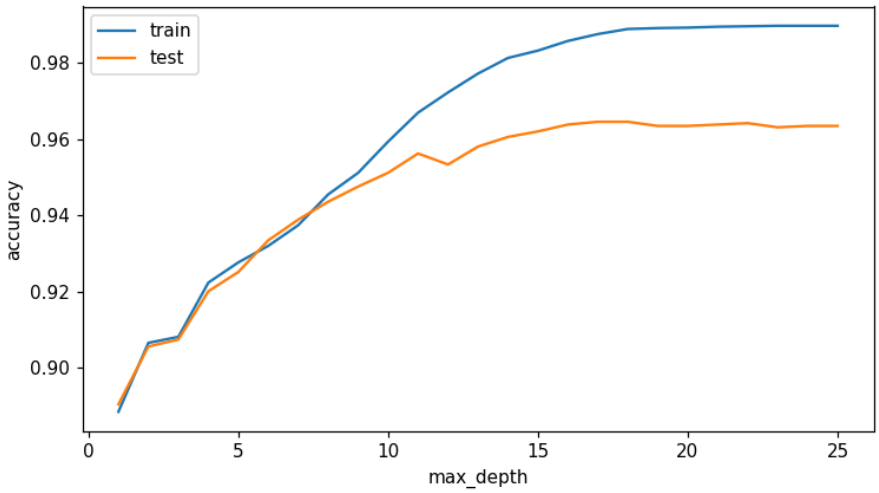


рис.2

Перевагою дерева є інтерпретованість: дерево глибини 3 (рис.3) вже дає зрозумілі правила – сайт без HTTPS із підозрілими посиланнями (AnchorURL) майже завжди фішинговий.

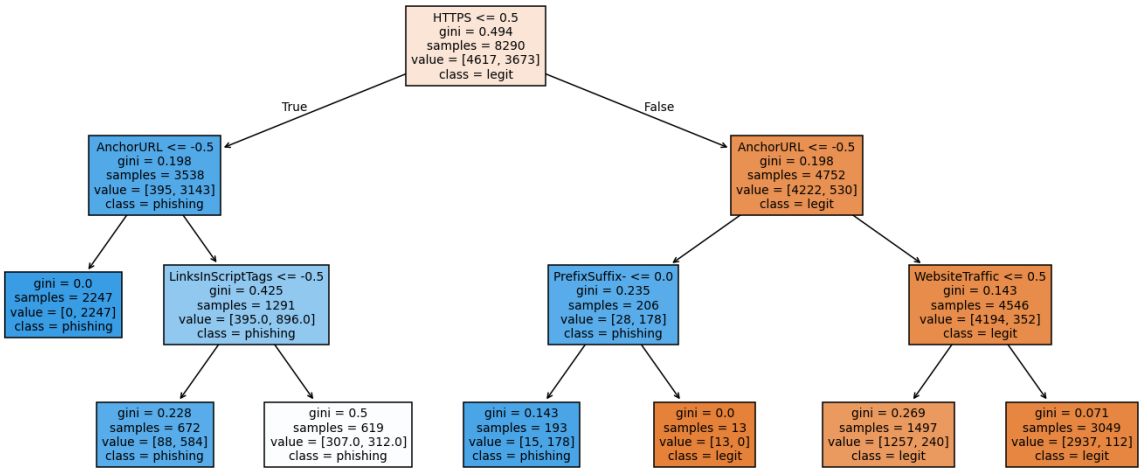


рис.3

NSL-KDD: при train accuracy ≈ 1.0 тестова точність падає до 0.77–0.79. Обидві моделі виявляють лише 23–38 % атак нових типів, яких не було в навчальній вибірці, а більшість помилок – пропущені атаки (рис.1). Це ілюструє нездатність дерев до екстраполяції. При цьому ROC AUC лісу 0.961 значно вищий, ніж у дерева (0.808): ліс краще ранжує записи за ймовірністю атаки, і зниження порогу відсікання підвищило б виявлення атак.

Важливість ознак за випадковим лісом (рис.4): для фішингу визначальні HTTPS та AnchorURL, для NSL-KDD – обсяги переданих даних (src_bytes, dst_bytes) і прапорець з'єднання.

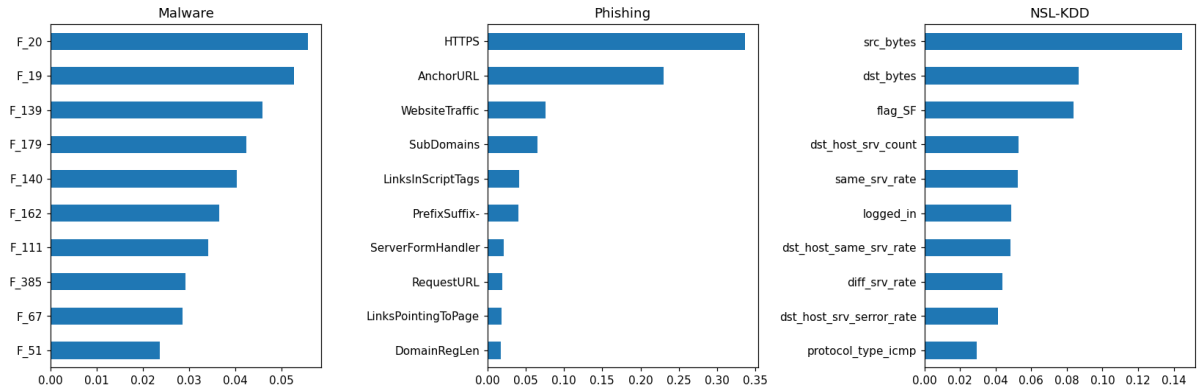


рис.4

5. Висновок

Переваги дерева рішень: простота, швидкість, наочні правила, відсутність потреби в нормалізації даних. Недоліки: перенавчання без обмеження глибини, нестабільність, нездатність до екстраполяції. Випадковий ліс зменшує перенавчання та дає стабільніші і точніші ймовірнісні оцінки (вищий ROC AUC на всіх датасетах), але втрачає інтерпретованість і потребує більше ресурсів.

Якість класифікації залежить насамперед від даних: на добре роздільних даних (Malware) обидва методи дають ≈ 100 %, на збалансованих даних середнього обсягу (Phishing) ліс стабільно кращий, а при зміні розподілу між навчальною і тестовою вибірками (нові атаки в NSL-KDD) якість різко падає незалежно від методу. Для виявлення невідомих атак класифікацію варто доповнювати методами виявлення аномалій.